Database II Lab/ 3rd Grade

[Second Semester] and [2026]

[Lab 9]

[28/4/2026] [8.30 + 10.30 am]

Instructor Information

Instructor
Dr. Rasool Hisham
Dr. Zainab Namh
Dr. Azhar Flaih
Assist. Prof. Zahraa Jaaz
Email Hours

[rasool.hisham@nahrainuniv.edu.iq] [2 Hrs]

Creating RDLC Reports in C# with Visual Studio

Lab Objectives

By the end of this lab, students will be able to:

- Understand what RDLC reports are and when to use them.
- Explain WHY a DataSet is required for RDLC instead of a DataTable alone.
- Create a DataSet (.xsd) with a DataTable that defines the report's field structure. • Design an RDLC report layout and bind fields from the DataSet.
- Write C# code to fill a DataTable with data from MySQL and display it in the report.

What is RDLC?

RDLC stands for **Report Definition Language Client-side**. It is a Microsoft reporting technology that allows you to create professional reports (tables, charts, headers, footers) inside .NET applications **without requiring a report server**. The report runs entirely on the client machine.

RDLC reports are typically used for printing structured data, generating invoices, employee lists, grade sheets, and similar documents.

The data flows through these components in this order:

Step	Flow
1	MySQL Database ⟶ Your C# code runs a SELECT query
2	C# code fills a DataTable (in memory) with the query results
3	You wrap the DataTable in a ReportDataSource and attach it to the ReportViewer
4	The ReportViewer reads the .rdlc layout file and renders the data on screen

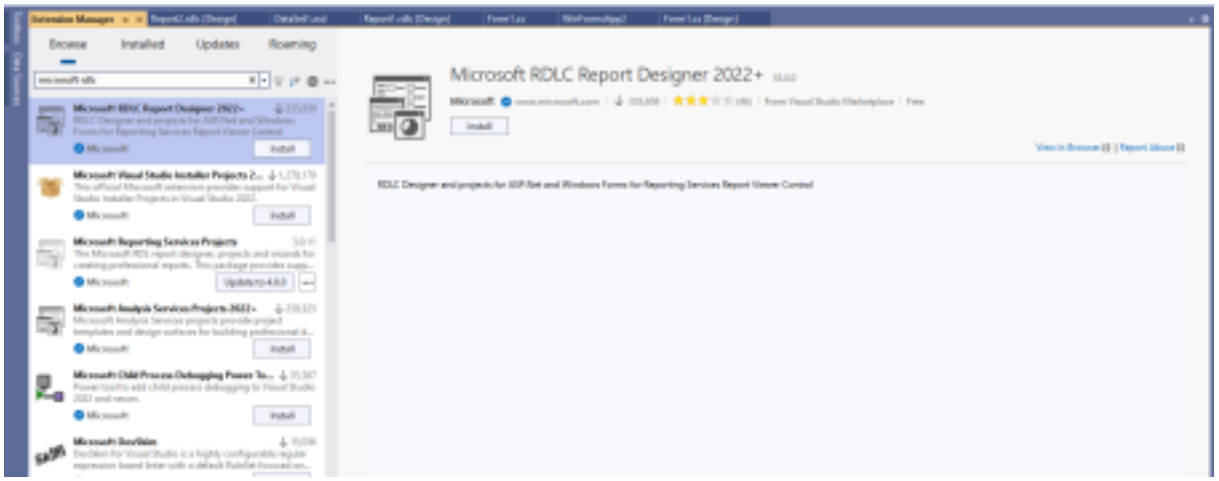
Create RDLC (Report Definition Language Client-side)

Step 1: Install the required Extensions

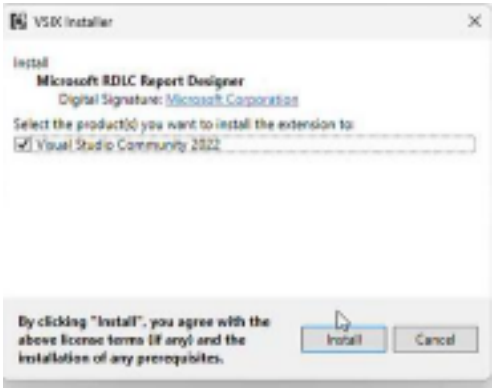
Microsoft RDLC Report Designer Extension enables **graphical design of RDLC reports** (*.rdlc files) directly inside Visual Studio. It brings in a **drag-and-drop interface** for creating professional-looking reports—tables, charts, headers, footers, data bindings, and more.

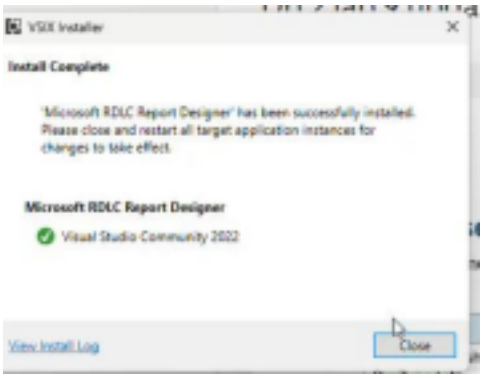
It is just like "Microsoft Word for Reports" inside Visual Studio.

1. Go to **Extensions** → **Manage Extensions** and browse for **Microsoft RDLC Report Designer** this extension will enable adding report (*.rdlc)



2. After clicking Install**, the Microsoft.RdlcDesigner.vsix** file will be downloaded**. Close Visual Studio,** then run the downloaded VSIX installer to complete the installation.





Note: VSIX stands for Visual Studio Integration Extension

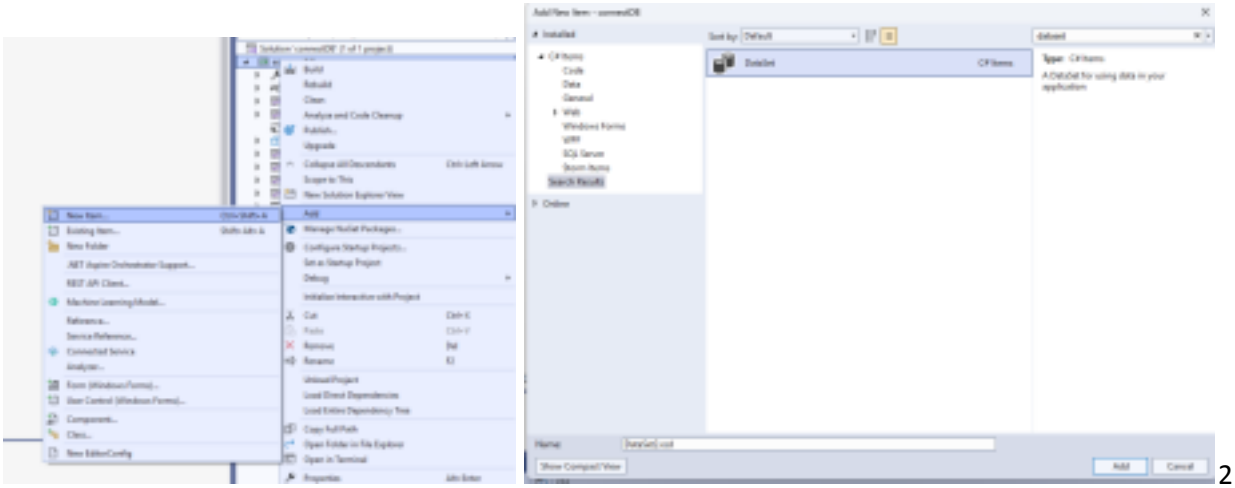
Step 2: Create a Dataset

1. Add a New Dataset:

A) Right-click on the **Project name** in the **Solution Explorer**.

B) Select **Add** → **New Item** → **Dataset**.

C) A dataset will be created that serves as the structure for your data.



2

2. Add Data Table:

A) Inside the Dataset, R-click inside it and add a **new DataTable**.

B) Name the DataTable (e.g. ContactsTable).

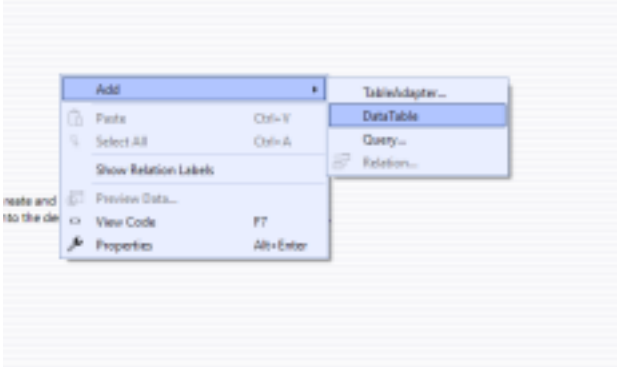
C) Right-click the DataTable and define the required **columns** (e.g., ContactName,ContactTel, …).

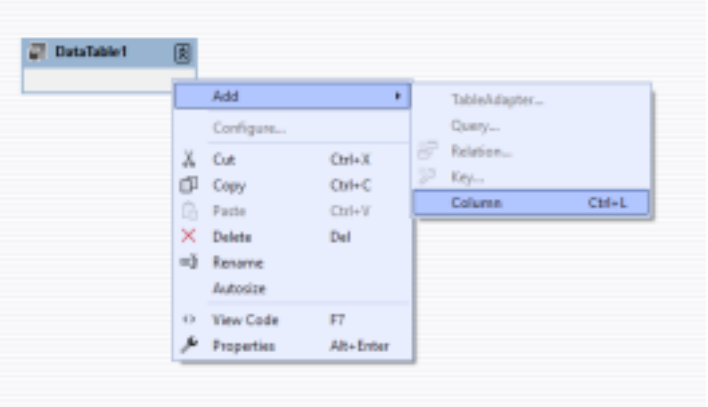
D) Save and close the dataset.

Why do we need this? The dataset acts as a ***container for data*** that you will display in the RDLC report. It is required to ***structure the data*** you intend to visualize and to provide it to the report.

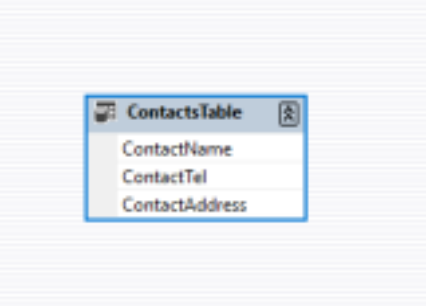
Why these specific names?
These column names MUST match the column names (or aliases) returned by your SQL query at runtime.

In this window, include the necessary columns for the report.





This is the final report fields:



Step 3: Create a New Report

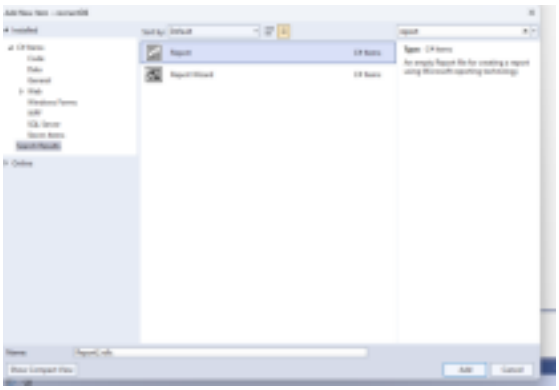
1. Add a Report:

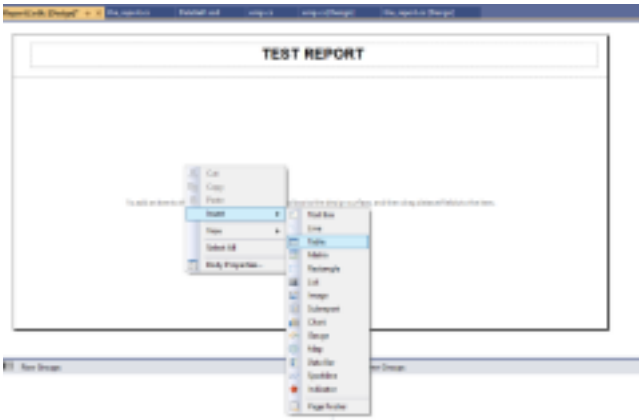
A) Right-click on the **Project name** again.
B) Select **Add** → **New Item** → **Report**. This will add a new *.rdlc file to your project.
C) Set the report name and click the **Add** button.

2. Design the Report:

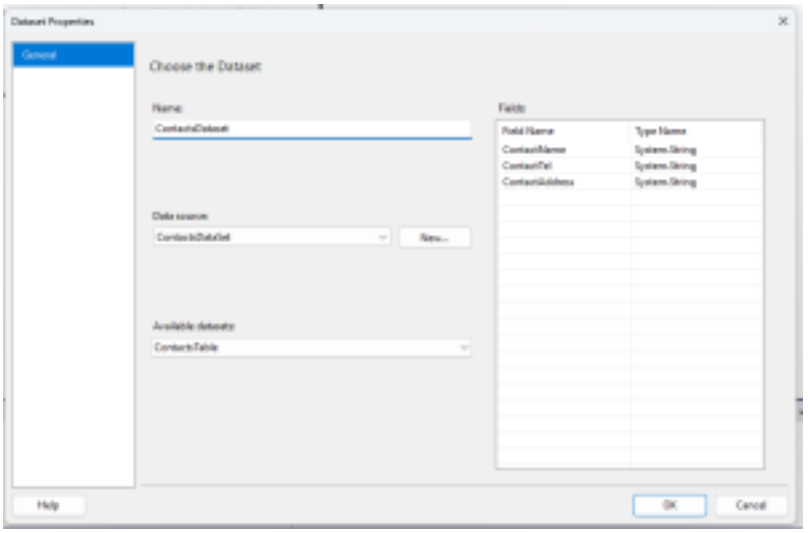
A) In the RDLC file, go to the **Design View**.
B) Add **fields** to the **Body** section to represent your data (e.g., ContactName, …).
C) Add a **Textbox** in the **Header** to show the title or other static information.

Why do we need this? The report file (RDLC) contains the ***layout and structure of the report***, defining where the data will be displayed, how it will appear, and the layout of tables, charts, or any other visual elements.





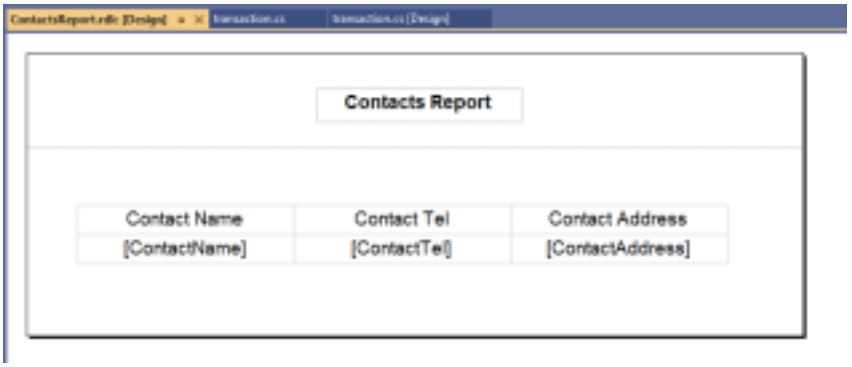
When adding the table, the dataset properties is shown:



Define the dataset name, data source and the datatable.

Dataset Name is the internal name for use within the report. C# code use this name as the source of the report. **Dataset Source** is your (.xsd file). The DataSet created in the previous step.

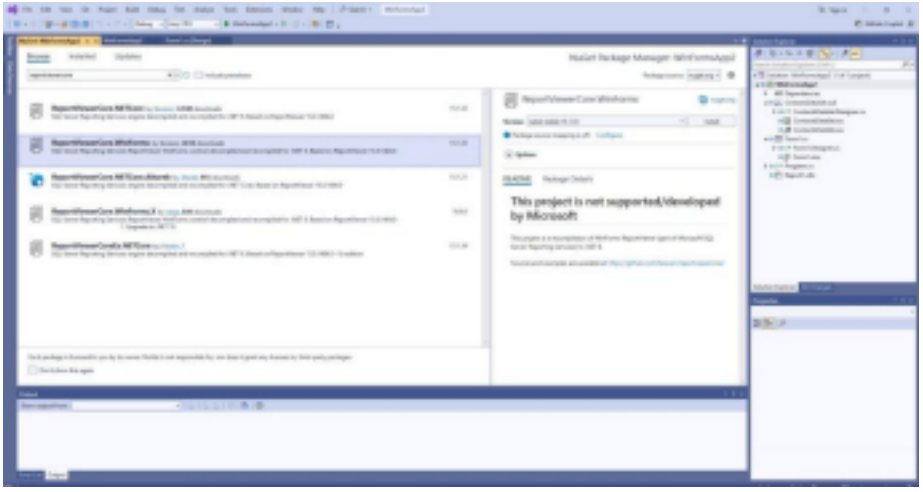
Then add the required fields to the report table:



Step 4: Add the ReportViewer Control and Bind Data to the Report

1. Install the NuGet Package:

1. Right-click on the **project name** in Solution Explorer and select **Manage NuGet Packages**.
2. In the Browse tab, search for **ReportViewerCore.WinForms**.
3. Select **ReportViewerCore.WinForms** and click **Install**.



5

2. Add the ReportViewer Control Programmatically:

A) Open your Windows Forms application.

B) the ReportViewer control is **not available in the Visual Studio Toolbox** for modern .NET projects. You must create it in code and add it to your form. This is done in the form's constructor or Load event.

C) **Add the namespace:**

At the top of your Form code (Form1.cs), add:

```
using Microsoft.Reporting.WinForms;
```

D) Bind Data to the Report (C# Code)

```
using System;
using System.Data;
using System.Windows.Forms;
using Microsoft.Reporting.WinForms;
using MySql.Data.MySqlClient;

namespace WinFormsApp2
 {
  public partial class Form1 : Form
  {
  private ReportViewer reportViewer1;
  // declared and assigned here
  private string connectionString =" ";

  public Form1()
  {
  InitializeComponent();

  // Create ReportViewer and add it to the form
  reportViewer1 = new ReportViewer();
  reportViewer1.Dock = DockStyle.Fill;
  this.Controls.Add(reportViewer1);
  }

  private void Form1_Load(object sender, EventArgs e)  {
  // 1. Clear old data
  reportViewer1.LocalReport.DataSources.Clear();

  // 2. Point to the .rdlc file
  reportViewer1.LocalReport.ReportPath = "ContactsReport.rdlc";
```

```
// 3. Get data from MySQL
DataTable dt = new DataTable();
using (var conn = new MySqlConnection(connectionString))  {
conn.Open();
string sql = "SELECT ContactName, ContactTel,  ContactAddress
FROM contacts";
using (var cmd = new MySqlCommand(sql, conn))  {
  var adapter = new MySqlDataAdapter(cmd); adapter.Fill(dt);
}
}

// 4. Bind data to report (match dataset name in .rdlc)  var rds =
new ReportDataSource("ContactsDataset", dt);
reportViewer1.LocalReport.DataSources.Add(rds);
```

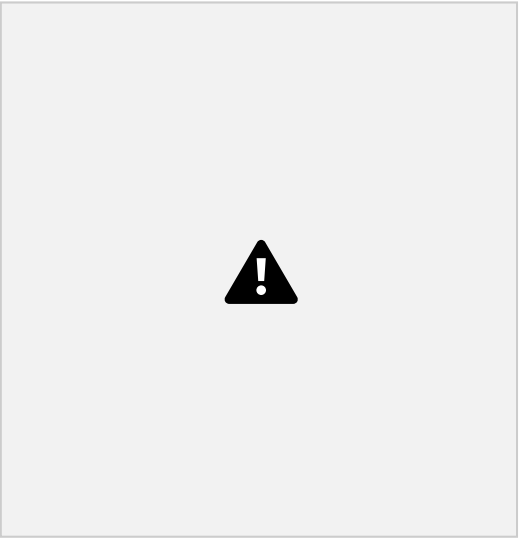


```
// 5. Show the report
reportViewer1.RefreshReport();
}
}
}
```

Line-by-Line Explanation

Line	What It Does	Why It's Needed
new ReportViewer() Dock = DockStyle.Fill Controls.Add(...)	Creates the ReportViewer control in code and adds it to the form, filling the entire form area.	In modern .NET, the control cannot be dragged from the Toolbox. It must be added programmatically.
DataSources.Clear()	Removes any previously attached data sources.	Clears existing data sources already added to the LocalReport.
ReportPath = "..."	Tells the ReportViewer WHERE the .rdlc layout file is.	Without this, the viewer has no layout to render — it doesn't know where to put the data.
DataTable dt + adapter.Fill(dt)	Runs the SQL query and stores the results in a DataTable in memory.	This is the ACTUAL DATA. The DataSet (.xsd) was just the design.
new ReportDataSource ("ContactsDataset", dt)	Used to bind data to the report. The constructor takes two parameters: DataSet and DataTable.	" ContactsDataset ": This is the name of the **dataset** or data source defined in the **RDLC** report. It must match the name defined in the report design. dt: This is the **DataTable** (dt), which contains the actual data that will be displayed in the report.

DataSources.Add(rds)	Attaches the data source to the report.	The report now has both a layout (.rdlc) and data (DataTable). It can render.
RefreshReport()	Triggers the ReportViewer control to refresh and render the report.	Without this, the viewer does not update. The data is attached but not displayed.

Important: Set "Copy to Output Directory" to "Copy always"

Set "Copy to Output Directory" to "Copy always" (as shown below) so the .rdlc file is automatically copied to the bin\Debug folder whenever you build — this way ReportPath = "ContactsReport.rdlc" can find the file using a simple **relative path** that works on any computer, not just yours.